

Tropical: Enhancing SLO Attainment in Disaggregated LLM Serving via SLO-Aware Multiplexing

Jinming Ma^{1*}, Jiefei Chen^{12*}, Xiuhong Li^{3#}, Jiangfei Duan¹⁴, Haojie Duanmu¹⁵

Xingcheng Zhang¹⁶, Chao Yang³, Dahua Lin¹⁴

¹Shanghai Artificial Intelligence Laboratory, ²Fudan University, ³Peking University

⁴The Chinese University of Hong Kong, ⁵Shanghai Jiao Tong University, ⁶Sensetime Research

Email: majinming@pjlab.org.cn, lixiuhong@pku.edu.cn

Abstract—To guarantee service quality in transformer based large language model (LLM) serving, it is essential to meet the latency constraints of both the *prefill* phase (measured by Time-to-First-Token, TTFT) and the *decode* phase (measured by Time-per-Output-Token, TPOT). *Non-disaggregated* serving places *prefill* and *decode* on the same worker, while *disaggregated* serving places the *prefill* and *decode* on isolated workers. However, no single architecture excels in both TTFT and TPOT metrics. After conducting a root cause analysis, we concluded that in *disaggregated* LLM serving, *prefill* execution has minimal interference with *decode* execution but result in high queuing times. In contrast, *non-disaggregated* LLM serving effectively reduces queuing times but introduces significant interference between *prefills* and *decodes*.

In order to leverage the best aspects of both *non-disaggregated* and *disaggregated* LLM serving, we have designed and implemented *Tropical*. *Tropical* introduces an service-level objectives (SLO)-aware multiplexing strategy that balances the queuing time and the interference, enabling the LLM serving to achieve high TTFT and TPOT SLOs simultaneously. Our evaluation of real-world datasets reveals that *Tropical* outperforms both state-of-the-art *non-disaggregated* and *disaggregated* LLM serving systems, achieving up to $2.09\times$ more requests within a 90% SLO attainment. Specially, compared to the *disaggregated* LLM serving system, *Tropical* improves P90 TTFT performance by $9\times$ with only an 15% reduction in P90 TPOT. Against the *non-disaggregated* LLM serving systems, *Tropical* delivers a $2.8\times$ performance improvement in P90 TPOT while maintaining the same P90 TTFT.

Index Terms—LLM Serving, SLO-Aware Scheduling.

I. INTRODUCTION

With the rapid development of the field of natural language processing, large language models (LLMs) [1]–[5] have become increasingly versatile. The transformer based LLM inference can be divided into two phases: the *prefill* and the *decode*. A request first complete *prefill* to generate the first token, and then process the *decode* repeatedly, generating a series of tokens in a streaming manner. In LLM serving, both phases have their own latency metrics that are highly relevant to user experience, which are time-to-first-token (TTFT) for *prefill* and time-per-output-token (TPOT) for *decode*. TTFT measures the time from when the user sends a request to when they first receive feedback. TPOT describes the average time for a single request to output tokens. As users typically expect to receive feedback quickly, optimizing TTFT is crucial. Besides, if TPOT is too long, users will experience delays while reading or processing the generations of LLM, which will affect the smoothness and continuity of the user experience. For a single request, both the TTFT and TPOT service-level objectives (SLOs) must be satisfied in LLM Serving.

LLM serving systems are categorized into *non-disaggregated* LLM serving systems and *disaggregated* LLM serving systems based on whether they separate two phases to isolated workers. *Disaggregated* LLM serving [6]–[9] use isolated workers to serve *prefill* and *decode* dividely. *Non-disaggregated* LLM serving systems [10]–[13] maximize

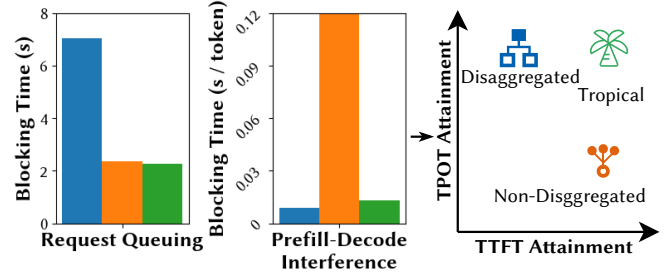


Fig. 1: Comparison among *Non-Disaggregated*, *Disaggregated* LLM Serving and *Tropical*.

GPU resource utilization by colocating *prefill* and *decode* process in the same worker. However, no single architecture excels in both TTFT and TPOT SLOs attainment. Because of high interference in *non-disaggregated* LLM serving and high queuing time in *disaggregated* LLM serving.

Non-disaggregated LLM serving experiences significant interference from *prefill* on *decode*. We evaluate the interference in Fig 1 (b) on a real dataset [8]. We quantize the interference by blocking time, the time a request in *decode* is blocked by other *prefill* executions. Interference in *non-disaggregated* serving is significantly higher than in *disaggregated* serving, leading to higher TPOT SLO violations. Instead, *disaggregated* LLM serving faces the mismatch between the allocation of *prefill-decode* workers and workload requirements. Because *prefill* can only use the resources of the *prefill* workers and cannot utilize the idle resources of the *decode* workers, the *prefill* workers are frequently under high workload, leading to longer queuing times for *prefill*. As shown in Fig 1 (a), the P90 queuing time for *disaggregated* serving is significantly higher than that for *non-disaggregated* serving. In *disaggregated* LLM serving, dynamically switching the role of *prefill* and *decode* is one of the strategies to address the issue of excessive *prefill* queuing times. However, the overhead of switching from *decode* workers to *prefill* workers is not negligible. Besides, the short-term volatility and unpredictability of LLM serving workloads also make it difficult for *prefill-decode* worker role switching to keep up with workload changes.

Based on our observations, we have found that collocating *prefill* and *decode* on the same worker in an SLO-aware manner can also improve the system's SLO attainment without switching overhead, which is called SLO-aware multiplexing. Based on the insight, we have designed and implemented *Tropical*. *Tropical* is a SLO-aware scheduler that manages multiple workers. *Tropical* introduces an multiplexing strategy that effectively reduces queuing time in the scenarios with highly dynamic *prefill* workloads. *Tropical* keeps the interference within an acceptable range that satisfies both TTFT and

* Equal contribution. # Corresponding author.

TPOT SLOs. As shown in Fig 1, *Tropical* achieves a Pareto optimality for TTFT and TPOT SLOs compared with both kinds of LLM serving.

In summary, this paper makes the following contributions:

- 1) We dive into the bottlenecks of *disaggregated* and *non-disaggregated* LLM serving and reveals that the main reason for the extended TTFT in *disaggregated* LLM serving is the queueing time. Conversely, the primary cause of the prolonged TPOT in *non-disaggregated* LLM serving is the interference.
- 2) We employ SLO-aware multiplexing to balance queueing time and interference, enabling the LLM serving to have high TTFT and TPOT SLOs attainment simultaneously.
- 3) Conduct a comprehensive evaluation of *Tropical* against state-of-the-art *non-disaggregated* and *disaggregated* LLM serving.

Our evaluation reveals that *Tropical* outperforms both state-of-the-art *non-disaggregated* and *disaggregated* LLM serving systems, achieving up to $2.09\times$ more requests within a 90% SLO attainment. Specially, compared to the *disaggregated* LLM serving system, *Tropical* improves P90 TTFT by $9\times$ with only an 15% reduction in P90 TPOT. Against the *non-disaggregated* LLM serving systems, *Tropical* delivers a $2.8\times$ improvement in P90 TPOT while maintaining the same P90 TTFT.

II. BACKGROUND

A. Prefill-Decode Scheduling

Disaggregated LLM serving has longer queueing times, while non-disaggregated LLM serving has more significant interference. We have drawn a timeline example to illustrate the reasons. Fig 2 (a) represents an *non-disaggregated* serving system [12], where prefills are scheduled eagerly once there are prefills in queue, resulting in a shorter queueing times, which means a shorter TTFT. However, the execution time for prefill is generally longer than that for decode. The execution of prefill will block the execution of decode, leading to a worse TPOT. State-of-the-art *non-disaggregated* LLM serving decompose long prefill into chunks. However, the execution of decode is still slow down because of the interference. Especially, the execution time of the chunks will significantly rise in the long context scenarios. Fig 2 (b) represents a *disaggregated* serving system, where requests in the *prefill* phase and the *decode* phase are executed on isolated workers. Requests in the *decode* phase are not interfered with by the other requests in the *prefill* phase, thus achieving a better TPOT. However, *prefill* can only utilize the resources of one worker, increasing queueing times and resulting in a worse TTFT. In addition, as shown in Fig 2 (b), the slower processing speed of the prefill phase also leads to idle time in the decode phase. Besides, since requests in the *prefill* and *decode* phases cannot be executed in the same batch, the computational resources of the GPU are not fully utilized, leading to *model FLOPs Utilization (MFU) inefficiency*.

B. SLO in LLM Serving

In this section, we define the TTFT SLO, TPOT SLO, and SLO attainment in the LLM serving. In addition, we define *slack* to describe the difference between the actual execution time and the SLO.

Generally, we define TTFT as the time from when the user sends a request to when they first receive feedback. Specially, we define TPOT as the total time spent generating tokens divided by the number of tokens generated, which equates to the average delay between tokens. We posit that users will only perceive waiting when the average generation rate surpasses their reading speed. Consider the following example: a LLM serving system rapidly delivers 20 tokens to the user, then encounters a 1-second pause before resuming token delivery. Although the maximum token-to-token delay for this request is 1

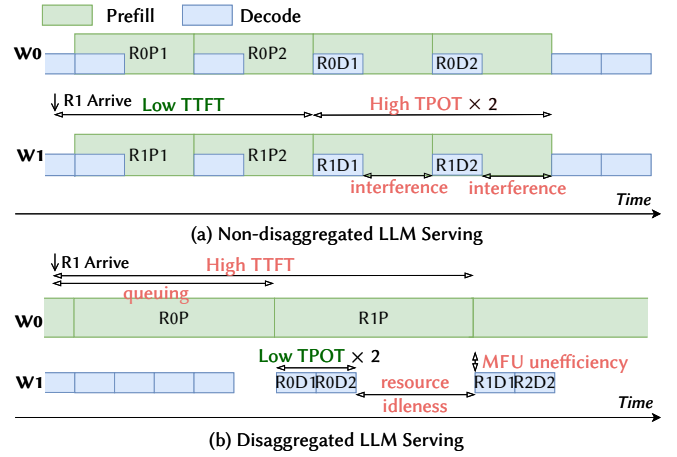


Fig. 2: Prefill-Decode scheduling comparison.

second, users still have a sufficient number of tokens to read during the pause, thus maintaining a positive user experience. Conversely, if users do not have enough information to process, they will perceive a lag in response. In *non-disaggregated* LLM serving, the TPOT can be broken down to the execution time and the interference caused by prefill execution. In *disaggregated* LLM serving, as the *prefill* and *decode* are executed in different workers, the interference of *disaggregated* can be composed into the decode execution and the migration time of KVCache from other prefill workers.

We define *slack* as the gap between system performance and its SLO. When the system's performance metrics exceed the SLO, this additional performance can be considered as slack, which provides a certain buffer, allowing the system to withstand a certain degree of load changes or performance fluctuations without violating the SLO. We utilize the slack during the *decode* phase to handle tasks from *prefill* phase without affecting the user experience.

Let $\mathcal{R}$ be the set of total requests r , L_r be the total generation lengths of r . The request SLO Attainment A is given by the following formulas:

$$\mathcal{R}_{TTFT} = \{T_{prefill}^r \leq TTFT_SLO \mid r \in \mathcal{R}\}, \quad (1)$$

$$\mathcal{R}_{TPOT} = \left\{ \frac{\sum T_{decode}^r}{L_r} \leq TPOT_SLO \mid r \in \mathcal{R} \right\}, \quad (2)$$

$$A = \frac{|\mathcal{R}_{TTFT} \cap \mathcal{R}_{TPOT}|}{\mathcal{R}}. \quad (3)$$

III. CHARACTERIZATION AND MOTIVATION

In this section, we investigate the characteristics of LLM serving to gain key insights and motivations that will guide our design and implementation of *Tropical*.

A. Workloads in LLM Serving

We observe the dataset [8] to analyze the workload of real-world LLM serving. Fig 3 (a) shows the total number of tokens arrived during an observation gap. It is evident that the arrival of tokens exhibits extremely high uncertainty in the short term. This uncertainty arises from two main factors. First, similar to other DNN serving [14], [15], the serving process itself is random, and user arrivals are unpredictable. Additionally, in the case of LLM serving, the number of input tokens for LLM serving also demonstrates significant dynamics. In Fig 3 (b), we present a scatter plot of a single request's (prefill, decode) sample, which illustrates the input uncertainty. In addition, it

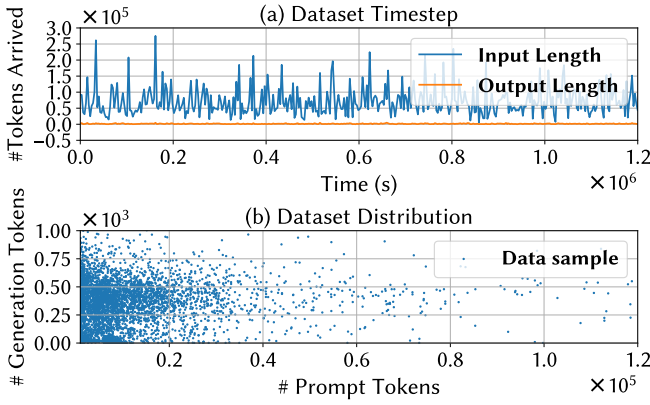


Fig. 3: Mooncake Dataset Token Lengths Over Time and Distribution.

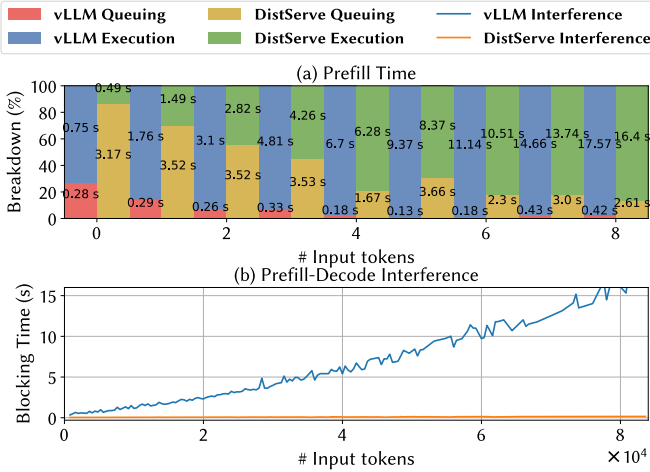


Fig. 4: Queuing and Interference Comparison of Non-Disaggregated Serving and Disaggregated Serving.

can be observed that the distribution of prefill text lengths follows a long-tail pattern. Both Fig 3 (a) and (b) indicate that, compared to the output, the input has a higher dynamic range during the LLM serving process. Similar conclusions can also be drawn from the dataset evaluated in other LLM serving-related works [7].

Characterization I: The lengths of prefill workloads in LLM serving exhibit high dynamism.

B. Efficiency of Non-Disaggregated and Disaggregated Serving

we conduct a quantitative analysis of both *non-disaggregated* and *disaggregated* scheduling strategies. In Fig 4 (a), we analyze the prefill time of the dataset [8] and provided a breakdown to observe the execution time, queuing time, and their proportions for different context lengths. We observed that when the text is short, the TTFT of the *non-disaggregated* LLM serving system is significantly better than that of the *disaggregated* LLM serving system. Although the decode phase in *non-disaggregated* LLM serving does interfere to the prefill, resulting in slightly higher execution times compared to *disaggregated* LLM serving, the queuing time in *non-disaggregated* LLM serving is much lower than that in *disaggregated* serving. This is particularly noticeable in scenarios with relatively shorter contexts, where the prefill time is dominated by queuing time, which is nearly $10\times$ longer than the execution time.

We observed that when the context is long, the TPOT of the *non-disaggregated* LLM serving system is significantly worse than

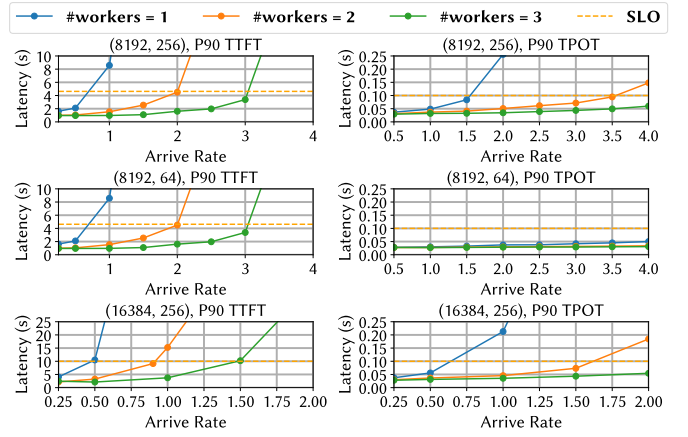


Fig. 5: P90 Latency in Different Worker Resource Allocations.

that of the *disaggregated* LLM serving system. Fig 4 (b) illustrates the interference caused by *prefill* execution on *decode* phase. As the interference primarily arises from the migration of KVCache in *disaggregated* LLM serving and from the prefill execution in *non-disaggregated* LLM serving. With the support of high-bandwidth KVCache transmission, the migration latency is significantly lower than that of prefill execution time. Therefore, the interference in *disaggregated* LLM serving is much less than that in *non-disaggregated* LLM serving. It is also evident that as the text length increases, the effectiveness of *disaggregated* LLM Serving in mitigating *prefill-decode* interference becomes more pronounced.

By combining the insights from Fig 4 (a) and (b), we observe that when the context length is relatively short, the queuing time in *prefill* phase significantly larger than the interference to *decode* phase. This means that, for short texts, the benefits of *disaggregated* LLM serving to mitigate interference are likely outweighed by queuing time.

Characterization II: The *disaggregated* architecture suffers from high queuing times but benefits from interference elimination. In contrast, the *non-disaggregated* architecture has lower queuing times but suffers from interference. Notably, as the length increases, the interference effects in the *non-disaggregated* architecture become more pronounced, while for shorter lengths, the proportion of queuing in the *disaggregated* architecture dominates.

C. Worker Resource Allocation in Disaggregated Serving.

With 4 worker resources, we examine different ratios of worker allocation under three (input, output) length configurations. Fig 5 shows the performance of P90 latency. We observed that it is difficult to match workloads and resources between *prefill* workers and *decode* workers simultaneously. With a configuration of (8192, 256), if we allocate 3 workers for *prefill* and 1 worker for *decode*, the LLM Serving can handle arrival rates of 3.0 for *prefill* and 1.6 for *decode*. In contrast, with 2 workers for *prefill* and 2 for *decode*, *prefill* can accommodate arrival rates of 2.0, while *decode* can handle up to 3.5. Fig 5 also shows the service capability of the LLM is constrained by both the *prefill* and *decode* capacities. According to the principle of the *leaky bucket effect*, static resource allocation for *prefill* and *decode* makes it difficult to achieve an optimal match between resources and workloads. When one phase is overloaded while the other remains idle, the system's performance can deteriorate significantly. In addition, due to the higher computational intensity, *prefill* requires more computing resources compared to *decode* as shown in Fig 5. Related works [7] shows the optimal ratio of *prefill-decode* can be up to 9:1. Besides, Fig 5 also reveals that different samples, such as (8192, 64) and

(16384, 256), have significantly different optimal ratios of prefill and decode workers.

Characterization III: Static ratios of *prefill* and *decode* worker allocation make it difficult to simultaneously match resources and workloads in both phases.

D. Challenge in LLM Serving

We summarize the challenges according to the observation.

Queuing and Interference balancing. From III-B, it can be observed that *non-disaggregated* LLM serving places *prefill* and *decode* on the same worker. However, the execution of *decode* is preempted by *prefill*, leading to Interference. Besides, if *prefill* and *decode* are separated and placed on different workers, *prefill* can only utilize a portion of the resources, causing *prefill* to potentially operate at a high service intensity or even overload, leading to a decline in performance by queuing. Existing LLM serving systems are unable to effectively balance queuing and interference times, making it difficult to simultaneously meet TTFT and TPOT SLOs.

Resource misallocation. According to III-C, even under fixed input-output length configurations, the ratio of prefill to decode can still lead to resource misallocation. From Section III-A, it can be concluded that the length of requests varies greatly. As a result, the *prefill-decode* ratio can also fluctuate significantly. To address the issue of mismatched resource allocation, an effective approach is to implement *prefill-decode* role switching in iteration level [16]. The overhead of role switching from *prefill* to *decode* is minimal, as it only requires requests that have completed the *prefill* phase to continue to the *decode* phase on the same worker, without the need for inter-worker transmission of KV cache. However, when the system performs the switching from *decode* to *prefill*, the system will find itself in a dilemma. Since the *decode* phase is stateful, immediately switching a *decode* worker to a *prefill* worker requires either migration or recomputation, both of which can lead TPOT SLO violations. If ignored, the *prefill* workers will experience overload, leading to the high queuing time. Consequently, achieving a rapid switching from *decode* workers to *prefill* workers is also quite challenging.

E. Optimization Chance

SLO-Aware Multiplexing. As described in II-A, We define *slack* as the gap between system performance and its SLO. When the system's performance metrics exceed the SLO, this additional performance can be considered as slack, which provides a certain buffer, allowing the system to withstand a certain degree of workload changes or performance fluctuations without violating the SLO. In LLM serving scenarios, we can utilize the slack during the *decode* phase to handle tasks from *prefill* phase without affecting the user experience. Whether performing *prefill-decode disaggregation* to eliminate interference or colocating *prefill-decode* into the same worker, the goal is to guarantee requests completion within the SLO. In a *non-disaggregated* LLM serving, *prefill* and *decode* sharing the same worker is akin to trading interference for reduced queuing time, while disaggregated LLM serving does the opposite. We term the practice of co-locating *prefill* and *decode* on the same worker while maintaining TTFT and TPOT SLOs as *SLO-aware multiplexing*. If *SLO-aware multiplexing* can be implemented in disaggregated LLM serving, it would be able to reach an equilibrium of queuing time and interference, which enhances the total SLO attainment.

IV. SYSTEM DESIGN

Based on the observations and the solution presented in section III, we design *Tropical*, an SLO-aware scheduler to balance the interference and queuing time.

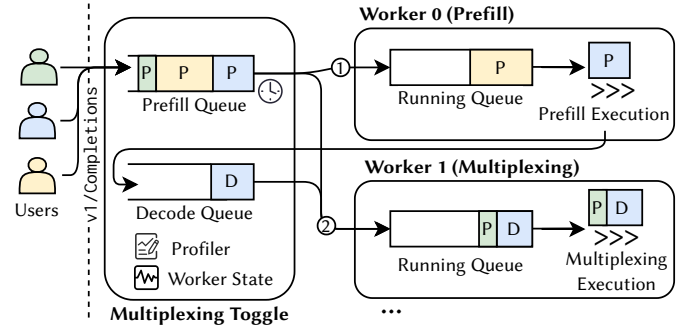


Fig. 6: Overview of *Tropical*.

A. Overview

The design of *Tropical* is illustrated in Fig 6. *Tropical* consists of two parts: the *multiplexing toggle* and a series of *workers*. In *Tropical*, workers can perform both *prefill* and *decode* execution. The phase in which they execute depends on the tasks sent to them by the multiplexing toggle. The multiplexing toggle primarily performs two functions: assignment and dispatching. For assignment, the multiplexing toggle assigns workers as prefill workers or multiplexing workers. For dispatching, It places incoming requests into the prefill queue initially and then dispatches them in an SLO-aware manner to competent workers. We will provide a detailed introduction to the design of the multiplexing toggle and its internal SLO-aware multiplexing mechanism in section IV-C and section IV-B.

B. SLO-Aware Multiplexing

As described in Figure 4, the *disaggregated* architecture suffers from high queuing times but benefits from interference elimination. In contrast, the *non-disaggregated* architecture has lower queuing times but suffers from interference. Additionally, as described in II-B and III-C, decode workers experience slack due to load fluctuations, and we aim to utilize the slack to serve prefills, thereby reducing the queuing time. In order to determine whether the interference from *prefill* execution will disrupt the requests that are currently in the *decode* phase, we will record the difference between the time taken for a single decode operation and the TPOT time, which is known as the TPOT slack in a single iteration.

If the predicted execution time is less than the slack time available in the *decode* worker, we treat the multiplexing worker as idle and schedule the prefill to the multiplexing worker to alleviate the pressure on the *prefill* workers. Although the insertion of *prefill* requests may lead to uneven output latency in decoding, recent advancements in LLM serving technologies [17], [18] aim to reduce fluctuations during the output process, ensuring a smoother flow. As described in Fig 7, the *decode* process to generate the third token of R_0 has accumulated enough slack to accommodate a prefill request. Therefore, *Tropical* routes a request from the prefill queue for execution. Although the prefill is inserted, R_0 does not violate the TPOT SLO.

Due to the significantly shorter execution time of short *prefill* than the long, it is more likely to route the short *prefill* request to multiplexing workers. As we concluded in 4, short *prefill* are queueing dominated, which gives *non-disaggregated* LLM serving an advantage. In contrast, long *prefill* cause greater interference with decoding, which makes *disaggregated* LLM serving outperform. SLO-aware multiplexing leverages the benefits of both serving strategy.

C. Multiplexing Toggle

As described in Fig 6, there are two paths for sending requests to workers. The multiplexing toggle provides a traffic control mechanism:

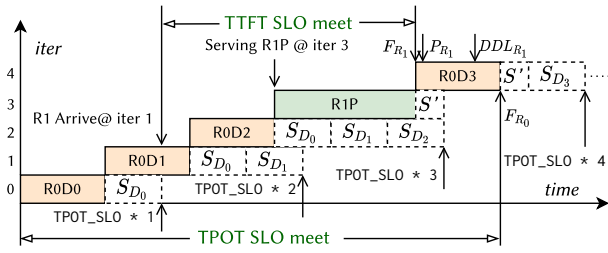


Fig. 7: P-D SLO-aware multiplexing by slack checking.

when the system is dominated by queuing, it follows Path ①, and when interference dominates, it follows Path ②. The multiplexing toggle can dispatch the request through Path ① to *prefill* workers for *prefill* execution, and then enter to the *decode* queue of the multiplexing toggle. The multiplexing toggle will dispatch the request again through Path ② to the multiplexing worker for *decode* execution. Alternatively, requests can bypass Path ① and directly enter the multiplexing workers. When the interference from *prefill* on *decode* remains within acceptable limits as described in IV-B, requests can be sent directly via Path ②. When the interference is not negligible, the *prefill* worker acts as an *interference inhibitor*, reducing the interference to an acceptable range. Since *prefill* can utilize the worker resources of both *prefill* and multiplexing workers, the system's queuing time will be significantly less than that of *disaggregated* serving.

The multiplexing toggle records the status of each worker, including monitoring the HBM watermark and the local queue of workers. Our design consolidates choice in Path ① and ②. Additionally, since the execution time of the LLM's *prefill* phase is highly predictable [19], we leverage offline profiling tools to estimate both the execution time of a *prefill* request and the queuing time when scheduling to the local worker. The multiplexing toggle conservatively sends requests to *prefill* workers only when the addition of the predicted execution time and the predicted queuing time in the workers, is less than the available slack to TTFT SLO. When a competent worker is found, an scheduling operator is created and requests are dequeued from *prefill* queue and enqueue to the running queue of the local worker.

Tropical also conservatively sends *prefill* requests to multiplexing workers. In addition to the TPOT slack budget outlined in IV-B, the multiplexing toggle records and predicts the execution time of the decoding batch for multiplexing workers. Given the high predictability of execution times, the multiplexing toggle will not route *prefill* requests to multiplexing workers when the execution time of the decoding batch is approaching the preset TPOT SLO. Additionally, since HBM usage is a critical factor affecting decoding execution, the multiplexing toggle will not select a multiplexing worker if the HBM usage for decoding exceeds the threshold. Furthermore, for multiplexing workers, we will also utilize chunked *prefill* to further smooth out the interference that *prefill* may cause to decoding.

When the watermark of all *decode* workers exceeds its threshold, indicating insufficient resources for *decode* workers, *prefill* workers can be directly assigned as multiplexing workers. If *prefill* frequently encounters delays (such as when *prefill* texts are too long), this will also trigger a transition from multiplexing workers back to *prefill* workers for a higher *prefill* throughput.

V. EVALUATION

In this section, we evaluate the efficiency of *Tropical*.

A. Experimental Setup

Models. We choose InternLM-20B [20], an LLM that is popular and performs exceptionally well in long context scenarios, the max supported context window is 200K, which satisfies the requirement of the maximum number of tokens in the dataset.

Testbed. We evaluated *Tropical* on a server equipped with 8 NVIDIA A100 80GB GPUs, with 600 GB/s P2P bandwidth between GPUs.

workloads. We choose Mooncake [8]. It is obtained from the trace of real long context LLM serving.

Metrics. We use SLO attainment as the major evaluation metric. Same as previous relative LLM serving works [7], [21]. We set the TTFT SLO to $5 \times$ the execution time of the latency in the corresponding phase under light workload. The other metrics we are concerned with are TTFT and TPOT average and P90 latency. To find the sources of benefit for TTFT, we analyze the queuing time. And we also show the cumulative distribution function (CDF) of the TTFT and TPOT.

Baselines. We compare *Tropical* with the following LLM serving:

- **vLLM** [11] vLLM is a representative *non-disaggregated* LLM serving system widely used in both academia and industry. It colocates the *prefill* and *decode* computation at the same worker and struggles to meet TPOT SLO.
- **vLLM with chunked prefill** [12]. Breaking down the *prefill* phase into multiple smaller chunks, and executing only one *prefill* chunk per iteration [12]. This approach can shorten the time *prefills* interfere *decodes*. However, this method still cannot effectively control the interference of *prefills* on *decodes*. We set the chunk size to 2048 based on the profiling of the workload.
- **DistServe** [6]. We implemented *disaggregated* serving based on vLLM [11], executing *prefills* and *decodes* in isolated workers, which is consistent with the implementation in DistServe [6].

Both the baselines and *Tropical* divided the 8 GPUs into 4 workers, configuring the degree of tensor parallelism of each worker to be 2. To evaluate the efficiency of SLO-aware multiplexing, *Tropical* and DistServe all use 2 workers for *prefill* and 2 workers for *decode*.

For both baselines and *Tropical*, we adapt the dispatching strategy of InFaas [22] with LLM serving, where the global dispatcher sends requests to the workers with the fewest unfinished token counts. In *Tropical*, the multiplexing toggle keeps track of the slack budget for multiplexing workers. Under the condition of satisfying the SLO, the multiplexing toggle will trigger multiplexing to piggyback *prefill* with the *decode* requests for execution.

B. SLO Attainment

In this section, we compare the end-to-end performance of *Tropical* against the baselines on real world datasets. Fig 8 (a) illustrates that when we gradually increase the rate, more requests will violate the latency requirements and the SLO attainment decreases. Compared to baselines, *Tropical* has improved performance by serving 2.02x more users. From Fig 8, it can be observed that the performance of chunked-*prefill* is not optimal. This is due to the fact that as the chunk index increases, the time to compute a chunk also increases, which can probably lead to TPOT SLO violation. *non-disaggregated* LLM serving also has lower SLO attainment than *Tropical*. To evaluate that the SLO violation is caused by queuing delay or interference. We further break down the SLO violation into TTFT and TPOT violation, we show the Pareto frontier diagram in Fig 8 (b), analyzing the changes in TTFT SLO attainment and TPOT SLO attainment for different LLM serving systems with the arrival rate. We found that DistServe has worse TTFT SLO Attainment than others, indicating that the performance of *disaggregated* LLM serving is dominated by the queuing time in the *prefill* phase, while vLLM has higher TTFT SLO

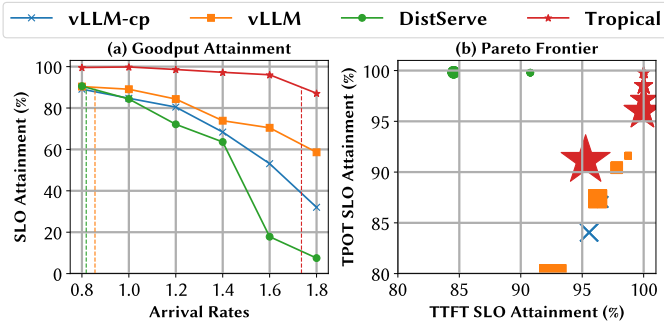


Fig. 8: SLO attainment.

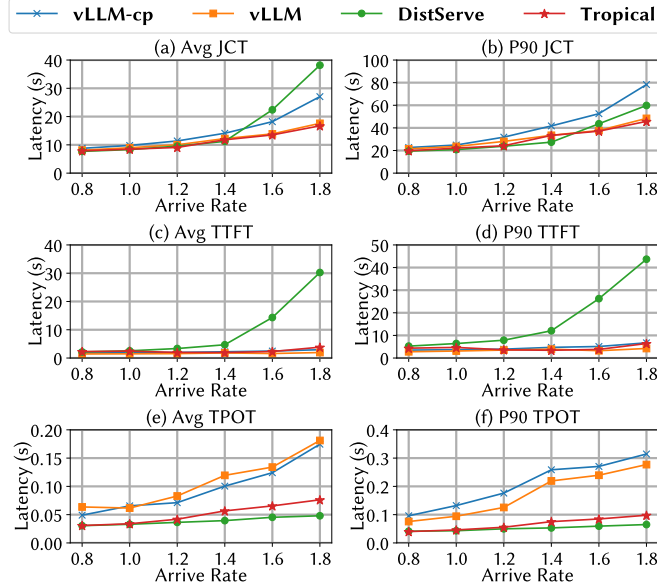


Fig. 9: Average and P90 Latency.

attainment, but compared to DistServe, its TPOT SLO is significantly reduced, suggesting that *non-disaggregated* LLM serving is dominated by *prefill-decode* interference. Evaluation show that the SLO-aware multiplexing can significantly reduce the TTFT queuing time and interference to ensure both TTFT and TPOT SLOs attainment.

C. Latency

As shown in Fig 9 (a) and (b), *Tropical* and vLLM outperform the other two serving systems for total latency. vLLM improves system worker resource utilization by co-location. *Tropical* achieves high worker resource utilization through SLO-aware multiplexing. *Tropical* improves TTFT metric by 9 $\times$ compared to DistServe. In 9 (e) and (f), the TPOT performance of *Tropical* is up to 2.33 $\times$ against vLLM and vLLM w/ chunked prefill. The TPOT performance of *Tropical* is slightly lower than DistServe. However, *Tropical* still meets the TPOT SLO for most requests as shown in Fig 8.

D. Queuing Time

To further analyze the sources of TTFT performance optimization against *disaggregated* LLM serving, we profile the queuing times of different LLM serving systems, as shown in Fig 10. *Tropical* improves P90 queuing time by 9 $\times$ compared to DistServe. The average queuing time and P90 queuing time of *Tropical* are slight higher than vLLM in high arrival rates. However, as shown in Fig 4, We use prefill workers to reduce the *decode-prefill* interference, therefore, the TTFT for *Tropical* and vLLM are essentially the same.

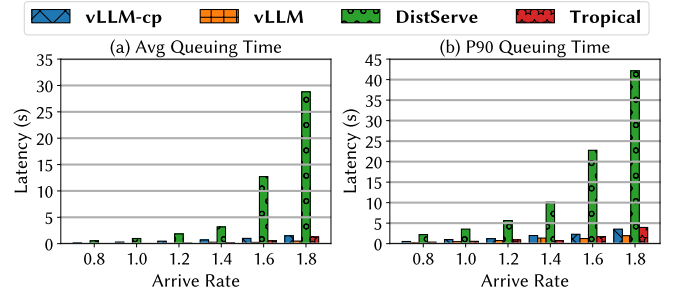


Fig. 10: Queuing Time.

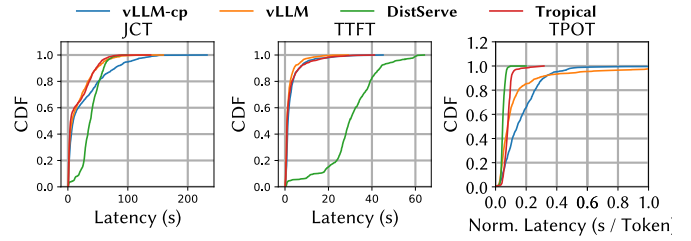


Fig. 11: CDF.

E. CDF

The CDF of serving is shown in Fig 11. It can be observed that, except for *Tropical*, all other serving systems exhibit significant tail latency. Instead, *Tropical* achieves a balance between TTFT and TPOT, thereby improving the overall service quality of the system.

VI. RELATED WORK

LLM Serving. The *non-disaggregated* inference service systems [10], [11], [13], [23], [24], which process the prefill phase and decode phase within the same worker. The *disaggregated* serving system [6]–[8], [25]–[29] handle the prefill phase and decode phase using different workers, eliminating the interference between requests. In the field of LLM serving, kernel optimization is a critical area of research. These optimization techniques demonstrate the diversity and depth of kernel optimization in LLM serving [30], [31]. In the context of Deep Neural Networks (DNNs), there are numerous SLO-aware scheduling strategies [14], [19], [22], [32], [33]. Andes [17] defines a Quality of Experience (QoE) metric and enhanced the service quality for users in the LLM serving scenario by optimizing this metric. DistServe [6] introduced the concept of Goodput and defined the TTFT SLO and TPOT SLO for the LLM serving service scenario. However, it did not consider maintaining a high resource utilization while simultaneously meeting TTFT and TPOT SLOs by SLO-aware multiplexing.

VII. CONCLUSION

We conducted a detailed analysis of the drawback of existing *disaggregated* LLM serving in TTFT by queuing time, and the deficiencies of existing *non-disaggregated* LLM serving systems in TPOT by interference. We design *Tropical*, a multi-worker scheduler. *Tropical* balances queuing time and interference by SLO-aware multiplexing, enabling the LLM serving to have high TTFT and TPOT SLOs attainment simultaneously.

ACKNOWLEDGEMENT

The authors sincerely thank anonymous DAC reviewers for their valuable comments on this paper. The research is supported by the Shanghai Municipal Science and Technology Major Project. Greatly thank Zihan Wang for his insightful feedback and constructive suggestions, which significantly improved the quality and conceptual framework of this work.

REFERENCES

- [1] Brown *et al.*, “Language models are few-shot learners,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [2] A. Chowdhery *et al.*, “Palm: scaling language modeling with pathways,” *J. Mach. Learn. Res.*, vol. 24, no. 1, Mar. 2024.
- [3] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [4] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [5] Vaswani *et al.*, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017.
- [6] Y. Zhong *et al.*, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 193–210. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yimin>
- [7] P. Patel *et al.*, “Splitwise: Efficient generative llm inference using phase splitting,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.
- [8] R. Qin *et al.*, “Mooncake: A kvcache-centric disaggregated architecture for llm serving,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.00079>
- [9] C. Hu *et al.*, “Inference without interference: Disaggregate llm inference for mixed downstream workloads,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.11181>
- [10] G.-I. Yu *et al.*, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/you>
- [11] Kwon *et al.*, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [12] A. Agrawal *et al.*, “Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 117–134. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [13] L. Zheng *et al.*, “SGLang: Efficient execution of structured language model programs,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=VqkAKQibpq>
- [14] H. Zhang *et al.*, “SHEPHERD: Serving DNNs in the wild,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 787–808. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/zhang-hong>
- [15] M. Shahrade *et al.*, “Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider,” in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 205–218. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/shahrad>
- [16] B. Wu *et al.*, “Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, ser. SOSP ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 640–654. [Online]. Available: <https://doi.org/10.1145/3694715.3695948>
- [17] J. Liu *et al.*, “Andes: Defining and enhancing quality-of-experience in llm-based text streaming services,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.16283>
- [18] B. Sun *et al.*, “Llumnix: Dynamic scheduling for large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 173–191. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/sun-biao>
- [19] A. Gujarati *et al.*, “Serving DNNs like clockwork: Performance predictability from the bottom up,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 443–462. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/gujarati>
- [20] Z. Cai *et al.*, “Internlm2 technical report,” 2024.
- [21] B. Wu *et al.*, “Fast distributed inference serving for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2305.05920>
- [22] F. Romero *et al.*, “INFaaS: Automated model-less inference serving,” in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 397–411. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/romero>
- [23] J. Duan *et al.*, “Muxserve: Flexible spatial-temporal multiplexing for multiple LLM serving,” in *Forty-first International Conference on Machine Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=R0SoZvqXyQ>
- [24] Z. Li *et al.*, “AlpaServe: Statistical multiplexing with model parallelism for deep learning serving,” in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 663–679. [Online]. Available: <https://www.usenix.org/conference/osdi23/presentation/li-zhouhan>
- [25] X. Miao *et al.*, “Spotserve: Serving generative large language models on preemptible instances,” ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1112–1127. [Online]. Available: <https://doi.org/10.1145/3620665.3640411>
- [26] H. Oh *et al.*, “Exegpt: Constraint-aware resource scheduling for llm inference,” ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 369–384. [Online]. Available: <https://doi.org/10.1145/3620665.3640383>
- [27] Y. Jin *et al.*, “P/d-serve: Serving disaggregated large language model at scale,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.08147>
- [28] F. Strati *et al.*, “Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving,” 2024.
- [29] C. Hu *et al.*, “Memserve: Context caching for disaggregated llm serving with elastic memory pool,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.17565>
- [30] T. Dao *et al.*, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.
- [31] K. Hong *et al.*, “Flashdecoding++: Faster large language model inference on gpus,” *arXiv preprint arXiv:2311.01282*, 2023.
- [32] M. Han *et al.*, “Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 539–558. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/han>
- [33] R. S. Kannan *et al.*, “Grandslam: Guaranteeing slas for jobs in microservices execution frameworks,” ser. EuroSys ’19. New York, NY, USA: ACM, 2019. [Online]. Available: <https://doi.org/10.1145/3302424.3303958>